

Projet

Nom du projet :	Projet Twizzy
Membre Projet:	Joly Timéo, Testa César, El Idrissi Wassim, Mehdaoui Hafsa

1. PRÉSENTATION DU PROJET

1.1 Contexte

Les constructeurs automobiles proposent aujourd'hui des technologies dites anti excès de vitesse permettant de lire automatiquement les panneaux de limitation de vitesse sur le bord des routes afin d'intégrer cette information sur le tableau de bord et/ou de la coupler avec les limiteurs automatiques de vitesse.

1.2 Objectifs du projet

Objectif principal :

L'objectif de ce projet est de programmer en Java à l'aide de classe le code de reconnaissance visuel des panneaux signalétiques et de travailler sur des bases de données afin de reconnaître les limitations. Ceci permettra d'adapter la vitesse du véhicule afin de répondre à la problématique sur les excès de vitesse.

Objectifs secondaires :

- Programmation par objets
- Bases de données et Traitement d'images
- Organisation en groupe

2. Organisation du Travail

Hafsa :Java

Wassim :Python

Timéo :Matlab

Cesar :Python

3. BESOINS ET EXIGENCES

3.1 Besoins fonctionnels

3.1.1 Fonctionnalité

Description : Java sur Eclipse

Critères :

La partie la plus importante de ce projet réside dans la bibliothèque Open CV 2.4.9 qui est la bibliothèque permettant l'extraction des images en tableaux de données (ici des couleurs).

3.1.2 Fonctionnalité 2

Description : Bases de données

Critères :

Ici, ce besoin d'avoir une base de données correctes n'a pas eu besoin d'être traité car nous avons déjà la base de données d'image et de vidéos des panneaux signalétiques et des vidéos à analyser.

4. CONTRAINTES

4.1 Contraintes temporelles

Date de début : Mercredi 18 Mars

Date de fin souhaitée : \\\

Jalons importants :

[Jalon 1] : 7 Mai, Rapport Mi-Parcours

[Jalon 2] : ////

4.2 Contraintes budgétaires

Aucune. Par ailleurs nous avons remarqué que pour certains ordinateur, selon que l'image est compliqué à analyser, les comparaisons d'images avec la partie ORB peuvent prendre plus de temps.

5. Ce qu'on a fait

Dans un premier temps, nous avons fait l'analyse avec matlab afin de comprendre comment notre projet allait fonctionner. Nous allons maintenant essayer d'adapter cela en Java avec des classes.

1) Puis nous avons extraits la base de données donné sur Arche, cela n'a pas été simple car le dossier est lourd avec de nombreuses images ne donnant rien car trop flou ou pas assez lumineux et codé en python (à peaufiner)

Par ailleurs, on a appliqué notre base de données a un réseau de neurone sur python (le même que le projet de classification des feuilles d'arbres) et on trouve une accuracy de 0,46 ce qui n'est pas très bon pour travailler avec cette base.

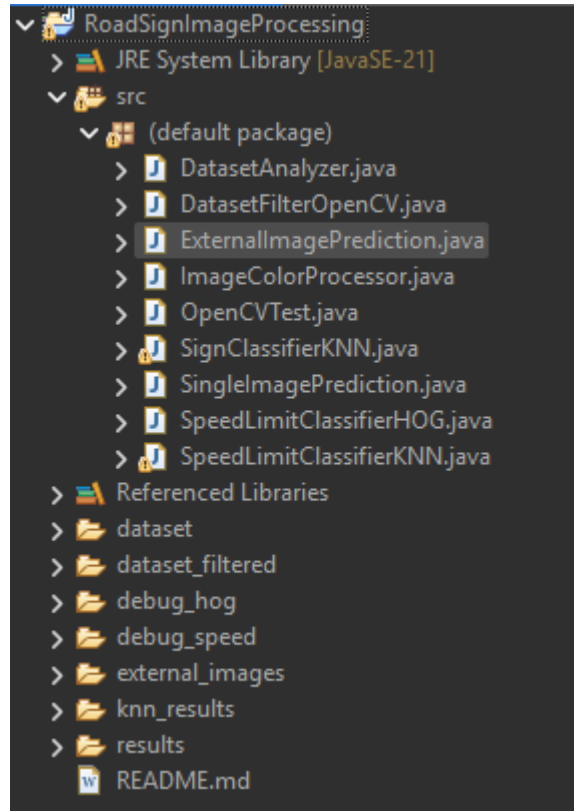
```

TensorFlow <= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please
use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
319/319 ————— 78s 241ms/step - accuracy: 0.2469 - loss: 0.0664 - v
al_accuracy: 0.2325 - val_loss: 0.0575
Epoch 2/10
319/319 ————— 75s 235ms/step - accuracy: 0.3664 - loss: 0.0432 - v
al_accuracy: 0.3109 - val_loss: 0.0503
Epoch 3/10
319/319 ————— 77s 241ms/step - accuracy: 0.4546 - loss: 0.0331 - v
al_accuracy: 0.3454 - val_loss: 0.0476
Epoch 4/10
319/319 ————— 78s 243ms/step - accuracy: 0.5446 - loss: 0.0228 - v
al_accuracy: 0.3740 - val_loss: 0.0494
Epoch 5/10
319/319 ————— 79s 246ms/step - accuracy: 0.6070 - loss: 0.0146 - v
al_accuracy: 0.4124 - val_loss: 0.0522
Epoch 6/10
319/319 ————— 77s 241ms/step - accuracy: 0.6403 - loss: 0.0100 - v
al_accuracy: 0.4434 - val_loss: 0.0563
Epoch 7/10
319/319 ————— 118s 369ms/step - accuracy: 0.6543 - loss: 0.0070 -
val_accuracy: 0.4810 - val_loss: 0.0611
Epoch 8/10
319/319 ————— 92s 290ms/step - accuracy: 0.6701 - loss: 0.0050 - v
al_accuracy: 0.4708 - val_loss: 0.0665
Epoch 9/10
319/319 ————— 77s 241ms/step - accuracy: 0.6787 - loss: 0.0039 - v
al_accuracy: 0.4606 - val_loss: 0.0779
Epoch 10/10
319/319 ————— 78s 243ms/step - accuracy: 0.6803 - loss: 0.0033 - v
al_accuracy: 0.4630 - val_loss: 0.0807

```

- 2) Par la suite, on a eu un problème (expliqué par la prof), sur la version de Eclipse qui n'est pas en V11 ou V17 pour les dernières versions de DeepLearning. On a donc essayé d'installer la version 11 sans succès sur les ordinateurs de l'école, j'essaierai sur mon ordinateur perso. Il s'agit dans cette partie de faire la transition de python vers java. Par la suite, avec la repartition du travail, nous avons bossé sur Matlab pour la partie qui va s'occuper de la base de donnée de la même forme que ce que nous avons fait lors de la classification de feuille d'arbre lors du s7 et du s8 en y incrémentant des filtres sur la base de donnée qui vont éliminer les images ou la moyenne de couleur est trop forte (sombre) ou trop faible (trop claire, saturée).
- 3) Pour la partie Java, on a commencé par mettre en place l'environnement Eclipse avec OpenCV afin de pouvoir lire et traiter les images directement depuis Java. Ensuite, on a travaillé sur la base de données fournie. Comme toutes les images n'étaient pas forcément exploitables, on a d'abord analysé les fichiers CSV présents dans les dossiers train, valid et test. Cela a permis de repérer les images avec un seul label, les images sans label et les images avec plusieurs labels. Après cette première analyse, un filtrage automatique a été mis en place. Le programme supprime les images non utilisables pour notre cas d'étude et crée une nouvelle base plus propre, organisée par classes, dans un dossier dataset_filtered. À partir de cette base filtrée, un premier modèle de reconnaissance a été entraîné avec OpenCV en utilisant la méthode KNN. Ce modèle général travaille sur plusieurs classes de panneaux et donne une précision d'environ 68 % sur la base de test. Comme certains

panneaux de limitation de vitesse étaient confondus entre eux, notamment à cause de leur forme très similaire, une deuxième méthode plus spécialisée a été ajoutée. Elle se concentre uniquement sur les panneaux de vitesse, extrait les chiffres du panneau, puis utilise HOG + KNN pour améliorer la reconnaissance. Cette méthode a permis de reconnaître correctement une image externe d'un panneau 60 km/h.



```
Chargement des images d'entraînement...
Images utilisées pour l'entraînement : 6864

Création des matrices OpenCV...
Entraînement du modèle KNN...
Modèle KNN entraîné avec succès.

Test du modèle...
Images testées : 519
Prédictions correctes : 354
Précision : 68.20809248554913 %
```

Un test sur cette image :



```
Chargement des images d'entraînement...
Images utilisées pour l'entraînement : 5263

Entraînement du modèle KNN spécialisé vitesse...
Modèle entraîné avec succès.

Prédiction de l'image externe...
-----
Image testée : 00003_00014_00019_png.rf.ed526e054bc87ed00ed23598ef72a0ac.jpg
Classe prédite : Speed_limit_60_km_h

Programme terminé.
```

6. A suivre

1) Python

Il va s'agir dans la suite du projet d'inclure un réseau de neurones convolutif pour la détection. Mais les chargements python de cette base sont relativement (TRES LONG) car nous n'avons pas encore filtrer les images, nous pensons à rajouter un filtre qui retire les images trop claires ou trop sombres $0 < 30$ et $220 < 256$. Par ailleurs, on a essayé d'appliquer un filtre sur les images avec de la reconnaissance et pour certains panneaux cela marche assez bien, mais pas pour les panneaux de 110 qui sont souvent confondu avec les 100. Un premier test de tri pour la base de données nous redonne 9000 images sur les 13000, mais cela reste à affiner.

Ici la partie RN nous donne une val_accuracy de 0.4, ce qui est beaucoup trop faible pour notre projet. Nous recherchons donc à implémenter une autre méthode comme celle de YOLO. Nous allons donc trier la base de données qui est beaucoup trop grande et qui contient des images qui ne permettent pas le bon apprentissage du modèle et essayer de le faire apprendre d'autres façons. Le réseau de neurones est donc à améliorer grandement.

```
Probabilités par classe :
Ahead_only: 0.0038
Beware_of_ice_snow: 0.0020
Bicycles_crossing: 0.0039
Bumpy_road: 0.0039
Children_crossing: 0.0032
Dangerous_curve_to_the_left: 0.0056
Dangerous_curve_to_the_right: 0.0028
Double_curve: 0.0062
End_of_all_speed_and_passing_limits: 0.0039
End_of_no_passing: 0.0030
End_of_no_passing_by_vehicles_over_3_5_metric_tons: 0.0027
End_of_speed_limit_80_kmh: 0.0028
General_caution: 0.0084
Go_straight_or_left: 0.0075
Go_straight_or_right: 0.0046
Keep_left: 0.0027
Keep_right: 0.0199
No_entry: 0.0040
No_passing: 0.0106
No_passing_for_vehicles_over_3_5_metric_tons: 0.0096
No_vehicles: 0.0060
Pedestrians: 0.0033
Priority_road: 0.0279
Right_of_way_at_the_next_intersection: 0.0047
Road_narrows_on_the_right: 0.0028
Road_work: 0.0082
Roundabout_mandatory: 0.0061
Slippery_road: 0.0054
Speed_limit_100_kmh: 0.0080
Speed_limit_10_kmh: 0.0006
Speed_limit_110_kmh: 0.0085
```

Prédiction finale : Speed_limit_70_kmh (0.19)

Ici on voit que la probabilité, ici avec une image d'un panneau signalisant la présence d'animaux sauvages, pour la meilleure classe est beaucoup trop faible, de plus est complètement fausse. Ce modèle n'est donc pas du tout satisfaisant pour l'instant.

La partie qui va nous servir à transférer du python au java n'est pas encore aboutie car il faut pour cela que le code python fonctionne absolument, or nous avons encore des choses à faire pour finir cela.

2) Java

La suite du travail consistera à améliorer la robustesse de la reconnaissance, notamment sur des images moins propres ou prises dans des conditions réelles. Il faudra aussi intégrer cette partie Java/OpenCV avec la suite du projet, notamment pour permettre ensuite le traitement d'images issues d'un flux vidéo.

Une amélioration possible serait aussi de tester d'autres méthodes de classification ou d'ajouter une étape de détection automatique du panneau dans une image complète, avant de lancer la reconnaissance.